

ETX 方案 v3：用同样的 SOTA kernel，融合后更快

作者：Kailun Chen · 状态：v3，取代 PLAN-moe.md 和 PLAN-fusion-win.md · 日期：2026-09-26 · 依据：设计文档 15.9–15.10 节，docs/RESEARCH-sync-hardware.md，docs/RESEARCH-moe-import.md

0. 结论先行

- **要证明的一件事：** 把生产环境真正在用的 kernel (vLLM 的) 原封不动放进 ETX 的一个持久化 megakernel，比同样这些 kernel 一个个 launch 更快。这样比较的只有“融合”本身，tile 写得好不好不参与比较。
- **已经做到的：** ETX 能原样导入 vLLM 的 kernel，GPU 上逐字节一致，完整模型 32/32 token 与 Hugging Face 一致。
- **还没做到的：** 融合版比同样的 kernel 逐个 launch 慢 7–16%。原因已经定位：ETX 的整卡同步比 kernel 边界贵。
- **修法：** 同步做三层处理。
- **少做：** 小 kernel 在每个 XCD 上各算一份，连续的小 kernel 合成一个任务。已写好。
- **做得便宜：** 跨 XCD 传递的小数据放进不经过 L2 的内存，每次同步不再需要写回和作废整个 L2。只动内存分配，不改 kernel。
- **等得快：** 每个 XCD 只留一个轮询者，调整休眠和优先级。
- **之后：** 在 MoE 模型上重复同样的比较，这是融合收益最大的场景；再扩展到小 batch 和多 GPU。
- **资源：** 单卡 MI300X 约 15 小时加 8 卡节点约 8 小时，前提是 GPU 随时可用，大约一周做完。当前余额 \$1.78。

1. 目标与验收标准

层级	标准
正确性（每一步都要满足）	每个导入的 launch 与 vLLM 原版逐字节一致（--check）；完整模型 32/32 token 与 HF greedy 一致
核心目标	同样的 kernel，ETX 融合版比 ETX 逐个 launch 快 至少 10% ，并且快于 vLLM custom_ops=all (vLLM 用同一组 kernel)
延伸目标	快于 vLLM 默认配置 (Inductor 生成的 norm/激活 kernel)；MoE 模型上快于 vLLM 最好成绩

2. 已有成果

导入工具 `etx/importer/`：

- 从 vLLM 源码里切掉 torch 的 host 代码，编成 AMDGPU IR。
- 把 `amdgpu_kernel` 改写成可内联的设备函数：参数从参数块读，block/线程 id 从 ETX 任务坐标换算，LDS 并进共用区。
- 一个 workgroup 里可以并排跑多个原版 block (slot)，各有自己的屏障。
- 自动生成 ETX tile 和 host 端的参数布局，再和设备库打成一个 bundle。

已接入 12 个 vLLM kernel： `wvSplitK`、`paged_attention_rocm` (mfma4/mfma16 两种加 reduce)、`rms_norm`、`fused_add_rms_norm`、`rotary_embedding`、`reshape_and_cache`、`silu_and_mul`。模板实例和 launch 形状都按 vLLM host 端的选择逻辑推出。

实测（MI300X, batch 1, 1024 上下文, ms/token）：

	Qwen2.5-1.5B	Qwen3-8B
vLLM 默认	1.862	4.906
vLLM custom_ops=all（同一组 kernel）	1.937	5.039
ETX 逐个 launch（同一份导入代码, HIP graph）	2.24–2.25	5.74
ETX 融合	2.61–2.62	6.16（2 个 token 的短测）*
ETX 融合, 小 kernel 固定到单个 XCD	2.58	—

* 8B 融合版完整 32 token 那次运行失败，没有输出，原因未查清（见第 6 节）。其余各行都是 32/32 token 正确；8B 的短测 2/2 正确。

3. 诊断：两个差距

差距 A：融合比逐个 launch 慢 0.4 ms（8B）。

- 每个 token 要过 310–472 个整卡事件，每个比一次 kernel 边界多花约 1.2 μ s。
- 根本原因在硬件：MI300X 的 8 个 XCD 各有一块 4 MB 的 L2，彼此不保持一致。ETX 的每个整卡事件都要生产者写回本 XCD 的 L2（buffer_wbl2 sc1），每个等待的 workgroup 再作废自己 L2 的非本地行（buffer_inv sc1）。kernel 边界每次 dispatch 只做一次，ETX 却是每个等待者都做一次。

差距 B：ETX 逐个 launch 比 vLLM 用同一组 kernel 慢 0.3–0.7 ms。嫌疑按可能性排：

1. ETX 自写的 argmax 旧写法：每个线程约 150 次相互依赖的加载。已改成向量化。
2. 小 kernel 被塞进同一个 1024 线程的 workgroup：attention 的 40 个 block 只占 10 个 CU，vLLM 是 40 个。已加 slot 数配置。
3. 导入后编译器按常量 blockDim 重新优化，生成的代码和 vLLM 的二进制不同。--check 已经能逐 kernel 计时对比。

4. 技术方案

4.1 同步：少做、做得便宜、等得快

手段	内容	状态
按 XCD 复制 (ETX_VL_XCD)	norm 和 SiLU 在 8 个 XCD 上各算一份（结果完全相同），写 XCD 本地的缓冲区；紧跟的 GEMV 排成 8×38 个任务，只等本 XCD 的事件。8B 的整卡事件从 472 个降到 254 个	已写好，本地编译零溢出，待测
合并小 kernel (ETX_VL_CHAIN)	q/k norm → RoPE → 写 cache 在一个任务里顺序执行，中间只隔 __syncthreads，每层少 3 个事件	已写好，待测
uncached 通信内存	跨 XCD 传递的激活向量（每层几 KB）和事件计数器放进 uncached 内存 (hipDeviceMallocUncached, MTYPE UC, 不经过 L2)。release 只剩 s_waitcnt vmcnt(0)，acquire 只剩作废 L1 (buffer_inv sc0)。权重运行中从不写，照常缓存。内存类型是页属性，kernel 不改	已写好（ETX_VL_UC, ETX_COHERENT_DEVICE_DATA；KV cache 的写入和读取固定在同一个 XCD），本地编译零溢出；是否运行由同步微基准判定

手段	内容	状态
每 XCD 一个轮询者 (relay, P5)	304 个轮询者降到 8 个。之前测出分层 acquire 不安全，是因为 L2 里可能留着陈旧的行；数据放进 uncached 内存后就没有陈旧行了	代码已有，要和上一项一起测
等待方式	s_sleep 先短后长（目前固定 8，约 0.2 μ s）；s_setprio 降低轮询 wave 的优先级；s_wakeup 提前叫醒同一 workgroup 里在睡的 wave	待调
硬件全局屏障 GWS	指令集里有：64 个资源，硬件排队，不用轮询。但 ROCm 在 gfx942 上自己也不用它，而且需要驱动给队列分配资源	只做一次微基准

4.2 追平 vLLM 的逐个 launch

用 --check 测出逐 kernel 的时间对比，确认差距 B 的构成，再用上面第 3 节的三项改动（argmax 向量化、slot 数配置、代码对比）逐项消除。

4.3 MoE (Qwen3-30B-A3B)

batch 1 时，每个 MoE 层 vLLM 只跑 6 个 kernel：

- router (wvSplitK)：已导入
- topk_softmax：HIP，可直接接
- 专家 gate_up：Triton fused_moe_kernel，192 个 program
- silu_and_mul：已导入
- 专家 down：Triton fused_moe_kernel，256 个 program
- moe_sum：HIP，可直接接

vLLM 在这个模型上只用到 26% 的带宽，是融合收益最大的地方。要补的有：

- Triton 的 IR 用 LLVM 23 生成，hipcc 是 LLVM 22。megakernel 的设备代码改用 LLVM 23 编（本地已在用这套），再用 hipModuleLoad 加载。
- 导入器支持 Triton 的写法：动态 LDS，以及末尾追加的 scratch 指针参数。
- Triton kernel 可能超过 128 个 VGPR 的上限：先从缓存读出它的实际用量；超了就用 512 线程的 megakernel。
- IR 要从 GPU 上 vLLM 实际跑过的 Triton 缓存里取：已加进上机脚本。

4.4 之后

- 小 batch (4-16) 和更长上下文：这是 serving 的常见场景。
- 多 GPU：把通信和计算融合在一起，对 kernel 边界来说这是最难的地方，megakernel 的优势最大。

5. 执行计划与 GPU 需求

阶段	天数	内容	完成标准	GPU
1	第 1-2 天	vm_next.sh：先跑同步微基准（bench/sync/sync_bench.hip：带数据和陈旧检查的 XCD 间往返、304 个 workgroup 的整卡屏障、kernel 边界间隔、GWS），按事先定好的门槛决定是否测 uncached 变体；再抓 vLLM trace 定天花板；然后逐 kernel 计时和各变体整模型	8B 和 1.5B 上，融合版比逐个 launch 快 10% 以上，32/32 正确	单卡约 3 小时

阶段	天数	内容	完成标准	GPU
2	第2-4天	MoE: 接入 topk_softmax 和 moe_sum; LLVM 23 设备编译 + hipModuleLoad; 导入 Triton fused_moe_kernel, --check 验证, 然后跑整个模型	Qwen3-30B-A3B 融合版快于 vLLM	单卡约 6 小时
3	第4-5天	batch 4-16, 更长上下文	每 token 时间优于 vLLM 同 batch	单卡约 6 小时
4	第5-7天	多 GPU: 张量并行/专家并行, 通信融合	8 卡上优于 vLLM	8 卡约 8 小时

合计: 单卡约 15 小时加 8 卡约 8 小时。本地能做的开发都放在上机之前完成, GPU 时间只用来验证和测量。

最理想的资源形式: 一台能随开随关的单卡 MI300X, 带持久磁盘, 模型和 vLLM 镜像不用每次重新下载, 省掉每次开机约 15 分钟的准备时间; 第 5-7 天再加一块 8 卡节点的时间。

6. 风险与应对

风险	应对
uncached 内存带来的延迟抵消了同步节省	只放每层几 KB 的向量和计数器, 先用微基准量化; 不行就退回”按 XCD 复制 + 合并”, 只保留 XCD 本地的同步
同步修好后, 融合仍然没有赢	那就说明 batch 1 dense 解码本身就是一条屏障链, 没有重叠空间 (之前手写 tile 时就这么判断过), 重点转向 MoE 和多 GPU, 那里有 kernel 边界做不到的重叠
Triton kernel 在 megakernel 里溢出寄存器	512 线程 megakernel; 或者只把 MoE 层单独做成一个 megakernel
8B 融合版完整运行失败 (未查清)	下次上机保留完整日志, 不再 grep 过滤; 看门狗超时时打印事件状态
GPU 拿不到	本地先把代码准备好; 每次上机按脚本一次跑完; 最好有专用机器

7. 已经准备好的东西

- 代码: github.com/metalopsccloud/amdproject (main 分支), 也同步在 [cxlxx/etx-megakernel](https://github.com/cxlxx/etx-megakernel)。
- 上机脚本: `examples/vllm_llm/vm_next.sh`。一次跑完两个模型、6 个同步变体、逐 kernel 计时对比、vLLM 的 kernel trace, 最后顺带采集 MoE 的 Triton 缓存。完整日志保留在 `~/vlnext/`。
- 调研: `docs/RESEARCH-sync-hardware.md` (同步的硬件手段)、`docs/RESEARCH-moe-import.md` (MoE 导入路线)。
- 设计文档: `docs/ETX_Technical_Design.md` 15.8-15.10 节 (导入方法与实测)。